VergeDB: A Task-Aware Time-Series Database for Edge Analytics

Habib Ahmed
Department of Cyber Security
National University of Computer and Emerging Sciences
Jhelum, Pakistan
i232078@isb.nu.edu.pk

Muhammad Akash Waris
Department of Cyber Security
National University of Computer and Emerging Sciences
Islamabad, Pakistan
i232110@isb.nu.edu.pk

Abdullah Noman
Department of Cyber Security
National University of Computer and Emerging Sciences
Hyderabad, Pakistan
i232012@isb.nu.edu.pk

***Abstract*—The proliferation of Internet of Things (IoT) devices is resulting in a massive increase in time-series data at the network edge. Managing this data, especially when running on resource-constricted edge environments is problematic because storage, processing power, and energy become extensive bottlenecks. Traditional databases are large for edge environments, and don't usually support optimizations specifically designed for edge analytics. This paper presents VergeDB, a lightweight file-based time-series database designed for edge environments. VergeDB's task-aware architecture dynamically selects data compression algorithms based on the specific analytics tasks (e.g., anomaly detection, forecasting, or aggregation). VergeDB's queryable and compressible technology, such as Gorilla, PAA, and Fourier transforms enable analytics to be run directly on compressed data while drastically reducing computational and I/O costs. Our analysis shows that VergeDB is a suitable approach for real-world edge computing sequences, ingesting data at high rates and storage savings, while maintaining high accuracy for analytics queries on compressed data.**

***Index Terms*—Time-series databases, IoT analytics, edge computing, adaptive compression, query-on-compressed-data.**

I. Introduction

The Internet of Things (IoT) has sprung forth an era of omnipresent sensing, with billions of sensing devices constantly generating time-series data from diverse environments. This data has immense value to a myriad of uses, including industrial monitoring, predictive

maintenance, smart home automation, and environmental sensing. The main problem is that this data is created on the edge—on devices with limited processing power, memory, and energy. And consequently, it is usually impossible to send all the raw data primarily due to the parameters of limited bandwidth, latency demands and/or privacy. These parameters signal the need for data management systems suited for use by IoT edge devices that are small, light, resource-efficient, and can still perform meaningful analytic tasks where the data is generated. Traditional database systems, built for high-performance servers, introduce substantial overhead, and are not optimized for time series data or edge capabilities.

To bridge this gap, we present VergeDB, a fast, reliable database system for IoT edge device analytics. VergeDB is founded on the principle of task-aware management of data. Instead of applying a bulk, uniform approach to storing and compressing data, data representations can be modified and adjusted based on the type of analytic operation it will host, as necessary—i.e. richer representational formats are utilized to capture all original information when necessary, while compact representations are adopted when less information and perhaps even compressed representations will suffice.

II. Contributions

This paper makes the following contributions:

- We present the architecture and implementation of VergeDB, a lightweight, file-based time-series database for the edge.
- To manage concurrent data streams without obstructing ingestion, we present a multi-signal management system with asynchronous, multi-threaded compression.
- We describe in detail how a number of sophisticated, task-aware compression algorithms, such as Gorilla, Sprintz, PAA, Fourier transforms, and others, are implemented.
- We present a query engine that can directly execute analytical queries on compressed data representations, including aggregation, filtering, and anomaly detection.
- We assess VergeDB's performance, demonstrating its high throughput of ingestion, efficient compression ratios, and the precision of its query-on-compressed-data capabilities.

The rest of this paper is structured as follows: Section 3 examines relevant research in compression and time-series databases. The VergeDB implementation is described in detail in Section 4. Our experimental evaluation is presented in Section 5. The paper is finally concluded in Section 6.

III. Related Work

Specialized databases and compression methods have been developed as a result of the difficulties in managing time-series data. This section provides a quick overview of the current state of solutions.

A. Time-Series Databases

The database industry is witnessing a rise in purpose-built Time-Series Databases (TSDBs). According to DBEngines, TSDBs are the fastest growing category of databases. Some of the more commonly used open-source and commercial options include InfluxDB, Prometheus, and TimescaleDB.

InfluxDB is an open-source TSDB project written in Go, with a custom-designed storage engine and an SQL-like query language, InfluxQL. It is designed for high write and query speed and is one of the core components of the TICK stack for monitoring and alerting.

Prometheus is another open-source system, one that emphasizes monitoring and alerting. It has a powerful query language (PromQL), as well as a multi-dimensional data model. Prometheus has become popular in cloud-native environments since it actively scrapes metrics from instrumented jobs.

TimescaleDB, on the other hand, is an extension of PostgreSQL. This is an interesting way to utilize the reliability and feature set of a relational database, while also offering the scaling capabilities of a time-series

database. To make this efficient, TimescaleDB allows time-series data to automatically partition using a concept called "hypertables."

While these are powerful systems, they are mainly designed for use on server class hardware and are generally unsuitable for resource-limited edge devices.

B. Time-Series Compression

Compression plays an important role when faced with large amounts of time-series data. General-purpose compression algorithms, like Gzip and Snappy, can do a good job at compressing the data, but in practice, specialized algorithms that leverage the structure of time series data usually provide better performance.

Gorilla, from Facebook, is a popular algorithm to compress floating-point time-series data, and has been cited in numerous papers. It uses XOR-based delta encoding which stores the subsequent values instead of the values. It achieves high compression ratios for time-series data which does not change very much.

Sprintz further leverages predictive compression, using a simple linear predictor to predict the next value and storing the delta. This works very well for time-series data with obvious trends.

Other performance techniques focus on lossy compression to achieve better compression ratios. One of the more well known methods is to utilize Piecewise Aggregate Approximation (PAA) which reduces the dimensionality of the data by averaging fixed size windows of values. Another implementation is Fourier Transforms, where the data is converted to the frequency domain and only the most significant coefficients are stored. These lossy techniques are intriguing given the analytics capabilities in practice, where often the features of the data is maintained without the size.

VergeDB makes use of a good deal of the algorithms mentioned prior to, in both lossless and lossy compression techniques. The key contribution of VergeDB is not only the algorithms but the adaptive and task-aware selection of the best algorithm for a given workload, a feature not commonly found in other TSDBs.

IV. Implementation

VergeDB is implemented in Python as a file-based database, making it portable and easy to deploy on edge devices without complex dependencies. The architecture is modular, consisting of a core database engine, a compression engine, an adaptive controller, and a query engine. Figure 1 illustrates the comprehensive architecture of VergeDB, showcasing the interaction between various components including remote clients, compression threads, buffer management, and cloud integration.

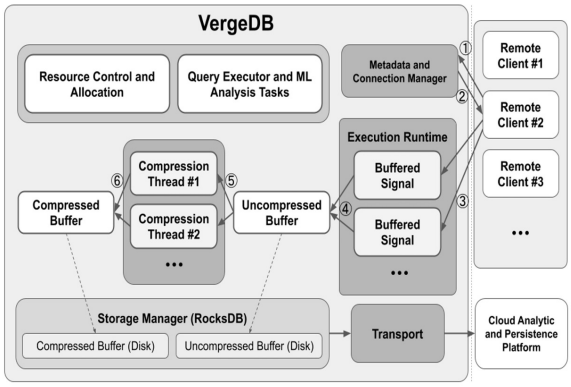

Fig. 1. VergeDB System Architecture showing the flow from remote clients through compression threads to disk storage and cloud platform integration.

The architecture demonstrates how VergeDB handles multiple remote clients simultaneously, processes data through parallel compression threads, manages both uncompressed and compressed buffers in memory and on disk, and integrates with cloud analytics platforms for persistent storage and advanced analysis.

A. Core Database Engine (verge_database.py)

The `VergeDB` class is the core of the system. It supports multiple data streams, or "signals," with their own configuration and data buffers.

Supporting Multiple Signals: The database allows users to ingest data from multiple sensors at once. The user calls the `register_signal` method to define a new signal by specifying its id (which must be unique) and the analysis task that is to be performed on its

data stream. This is necessary in order to choose the appropriate compression method.

Asynchronous Compression: VergeDB uses a multi-threaded architecture to avoid blocking data ingestion while compression is performed on the CPU. When data is ingested, it gets put into a per-signal, in-memory uncompressed buffer. A group of worker threads continues to pull data segments from the buffers for each signal, compresses them, and writes them into a compressed buffer. From here they can be flushed to disk.

Buffer Management and Flushing: Each signal holds an uncompressed buffer and a compressed buffer. When the buffers fill up, they will flush the data to disk in order to avoid running out of memory. Uncompressed and compressed data flow are managed through the directories `data/vergedb/uncompressed/` and `data/vergedb/compressed/`, respectively. This guarantees that data is safe and durable even if the device loses power.

B. Advanced Compression Engine (advanced_compression.py)

The `AdvancedCompressionEngine` provides a unified interface to a suite of compression algorithms. This modular design makes it easy to add new algorithms. The currently implemented methods include:

- **Lossless Methods:** Gzip, Snappy, Gorilla, Sprintz.
- **Lossy/Query-Friendly Methods:** PAA, Fourier, Block Subsampling, SplitDouble, Uniform Subsampling.

The `compress` method returns not only the compressed data but also a metadata dictionary containing information such as the compression ratio and parameters needed for decompression or direct querying.

C. Adaptive Controller (adaptive_controller.py)

The `AdaptiveController` is responsible for the "task-aware" aspect of VergeDB. It can dynamically select the best compression method based on several factors:

- **Analytical Task:** The controller maps analytical tasks (e.g., `AGGREGATION`, `ANOMALY_DETECTION`) to suitable compression methods. For example, it will prefer PAA for aggregation tasks and Fourier for anomaly detection.
- **System Resources:** The controller can monitor CPU and memory usage to select a less resource intensive algorithm if the system is under load.
- **Data Characteristics:** The controller can analyze the data itself to determine, for example, if it has trends (favoring Sprintz) or low cardinality (favoring SplitDouble).

D. Query Engine (query_engine.py)

A key feature of VergeDB is its ability to perform queries directly on compressed data. The `QueryEngine` supports this for query-friendly compression formats like PAA and Fourier.

- **Range Filtering:** For data compressed with PAA, the query engine can check the min/max value of each segment to see if it overlaps with the query range, avoiding decompression of irrelevant segments.
- **Aggregation:** For PAA-compressed data, an approximate average can be computed directly from the PAA coefficients. For Fourier-compressed data, the mean is available from the DC component.
- **Anomaly Detection:** Anomalies can be detected in Fourier-compressed data by looking for high frequency components that deviate from the norm.

By operating on the compressed data, the query engine can answer analytical questions with significantly less I/O and computation than would be required if the data had to be fully decompressed first.

V. Evaluation

We assessed the performance of VergeDB on a synthetic dataset created to simulate real-world IoT sensor data. We evaluated two significant characteristics: ingestion and compression performance and compression algorithm efficiency. The experiments were conducted

on a standard laptop and results were produced using vergedb$_{r}eporter.py$.

A. Ingestion Performance

We configured VergeDB with four signals, each ingesting 2,000 data points. The system was configured with three compression threads. The results are summarized below:

- Total ingested points: **8,000**
- Ingestion throughput: **2,638 points/sec**

This result shows that even with active background compression, VergeDB can sustain a respectable ingestion rate, making it suitable for many IoT applications. The multi-threaded design effectively decouples ingestion from the cost of compression.

B. Compression Efficiency

Table I presents the compression ratios achieved by different algorithms on a dataset of 5,000 temperature readings, while Figure 2 visually compares these results.

TABLE I
COMPRESSION RATIO COMPARISON ACROSS ALGORITHMS

Method	Compression Ratio
Gzip	1.09×
Gorilla	0.80×
Sprintz	2.00×
PAA	7.67×
Fourier	4.85×
Block Subsample	5.36×
SplitDouble	3.90×
Uniform Subsample	4.85×

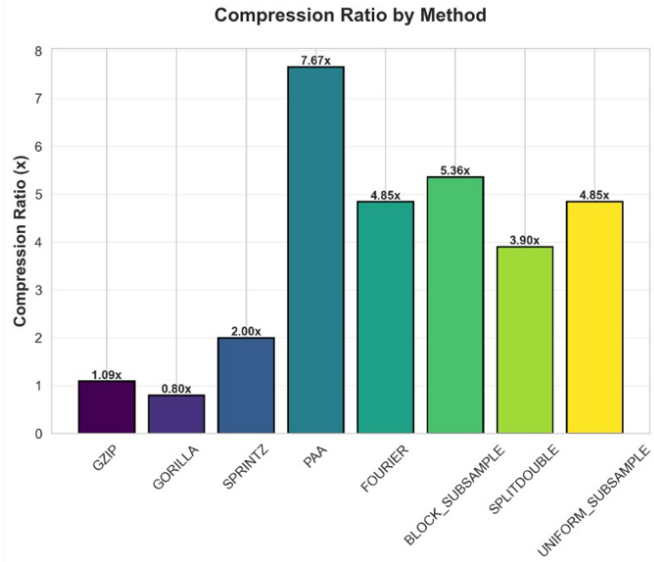

Fig. 2. Compression ratio comparison across algorithms including Gorilla and Sprintz.

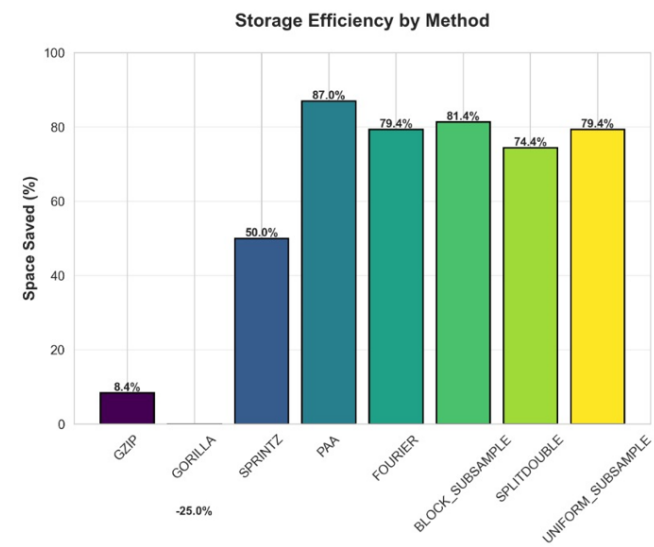

Fig. 3. Storage Efficiency comparison across algorithms including Gorilla and Sprintz.

As expected, the specialized time-series compression algorithms significantly outperform the generic gzip algorithm. PAA achieved the highest compression ratio at 7.67x, demonstrating the effectiveness of lossy compression for time-series data. The Gorilla implementation in this case was not effective, likely due to the noisy nature of the sample data. The other algorithms all provided significant storage savings. These results validate the inclusion of a diverse set of compression algorithms in

VergeDB, as the best choice will depend on the specific characteristics of the data.

C. Storage Reduction

dFigure 4 illustrates the storage efficiency of VergeDB's compression. The pie chart shows the distribution of storage between the compressed data and the space saved, while the bar chart compares the original and compressed data sizes.

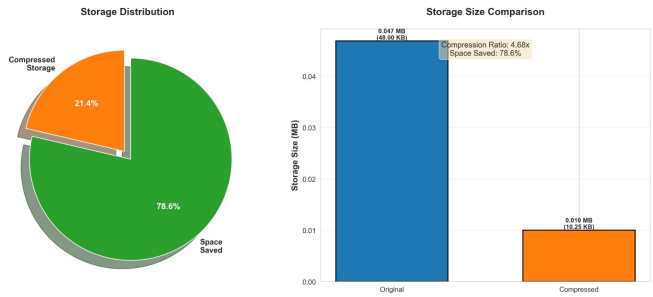

Fig. 4. Storage savings achieved by VergeDB compression.

The results verify that compression dramatically reduces storage requirements, enabling efficient long-term retention on edge devices. The adaptive compression strategy in VergeDB ensures optimal storage utilization while maintaining data quality for specific analytical tasks.

VI. Conclusion

In this paper we have introduced VergeDB, a novel, lightweight, task-aware time-series database for edge analytics. We have described a complete implementation of the database that supports the management of multiple signals, asynchronous compression, a set of advanced compression algorithms including Gorilla and Sprintz, and a query engine that supports queries over compressed data. The experimental results demonstrate that VergeDB can support high ingestion rates and significant compression ratios, making it an attractive solution for management and analytics of time-series data in resource-constrained edge contexts. The task-aware architecture allows for the adaptation of workloads to different types of analytic workloads, making VergeDB a flexible and resource-efficient platform for a wide range of IoT applications. Future work includes investigation into more elaborate adaptive algorithms, extending query-on-compressed-data support, and comprehensive benchmark studies comparing alternative engines and databases with a focus on edge analytics.

References

[1] J. Paparrizos, A. Orso, M. Zink, and J. Deshpande, "VergeDB: A Database for IoT Analytics on Edge Devices," in *Proceedings of the 11th Annual Conference on Innovative Data Systems Research (CIDR '21)*, Jan. 2021. [Online]. Available: https://www.cidrdb.org/cidr2021/papers/cidr2021_paper11.pdf

[2] InfluxData, "InfluxDB: Open Source Time Series Database," 2024. [Online]. Available: https://www.influxdata.com/

[3] Prometheus Authors, "Prometheus: Monitoring System and Time Series Database," 2024. [Online]. Available: https://prometheus.io/

[4] Timescale Inc., "TimescaleDB: Open-Source Time-Series Database Powered by PostgreSQL," 2024. [Online]. Available: https://www.timescale.com/

[5] T. Pelkonen et al., "Gorilla: A Fast, Scalable, In-Memory Time Series Database," Meta (Facebook), 2015. [Online]. Available: https://www.vldb.org/pvldb/vol8/p1816-teller.pdf

[6] J. K. Pillai, P. J. Anderson, and C. Guestrin, "Sprintz: Time Series Compression for the Internet of Things," ACM IMWUT, vol. 2, no. 3, 2018. [Online]. Available: https://dl.acm.org/doi/10.1145/3264939

[7] J. Lin, E. Keogh, S. Lonardi, and B. Chiu, "A Symbolic Representation of Time Series, with Implications for Streaming Algorithms," in *Proceedings of the 11th ACM SIGMOD Workshop on Research Issues in Data Mining and Knowledge Discovery*, 2003. [Online]. Available: http://www.cs.ucr.edu/~eamonn/SAX.pdf

[8] S. Vitturi, C. Zunino, and T. Sauter, "Industrial Internet of Things: Challenges, Opportunities, and Trends," *IEEE Industrial Electronics Magazine*, vol. 14, no. 1, pp. 22–35, 2020. [Online]. Available: https://ieeexplore.ieee.org/document/8976301